

8. Hazifeladat Bemutató

A data driftet szeretnénk merni!

Egy jupyter notebook segítségével új és régi borok minőségét jósoljuk meg az API-nkkal!

A régi borok a tanító halmazban lévő vörös boraink deduplikálva.

Az új halmaz tartalmazza a régi vörös és új fehér borokat is. Ez a halmaz nincs deduplikálva!

Ezt követően EvidentlyAI-jal a reportokat kimentjük és ezeket Streamlittel hostoljuk!

Walkthrough

Jupyter notebookból történő inference.

Problem Statement

We trained our model with the red wines dataset!

Let's assume that our customers want to classify both red and white wines with our model.

Let's generate an EvidentlyAI report to see the change.

```
df_red = pd.read_csv("../data/winequality-red.csv", sep=";")
df_white = pd.read_csv("../data/winequality-white.csv", sep=";")

df_red["target"] = np.where(df_red["quality"] < 7, 0, 1)
df_white["target"] = np.where(df_white["quality"] < 7, 0, 1)

df_red.drop("quality", axis=1)
df_white.drop("quality", axis=1)

reference = df_red.drop_duplicates()
current = pd.concat([df_red, df_white])
```

[2] ✓ 0.0s

```

# This needs the API running

# Let's get the prediction for all datapoints!
import requests

# Assuming `current` is your DataFrame
url = "http://localhost:8000/model/predict"
headers = {"accept": "application/json", "Content-Type": "application/json"}

for df in [reference, current]:
    predictions = []
    for idx, row in df.iterrows():
        # Convert row to dict, only include feature columns
        payload = row.to_dict()

        # Send POST request
        response = requests.post(url, headers=headers, json=payload)

        if response.status_code == 200:
            predictions.append(response.json()["predicted_quality"]) # store API output
        else:
            print(f"Failed for row {idx}: {response.status_code}, {response.text}")
            predictions.append(None)

    # Optionally, add predictions to your DataFrame
    df['prediction'] = predictions

```

[3] ✓ 26.8s

```

data_def = DataDefinition(
    classification=[BinaryClassification(target="target", prediction_labels="prediction")]
)

ref_ds = Dataset.from_pandas(reference, data_definition=data_def)
cur_ds = Dataset.from_pandas(current, data_definition=data_def)

report = Report(metrics=[
    DatasetStats(),
    DuplicatedRowCount(),
    AlmostDuplicatedColumnsCount(),
    AlmostConstantColumnsCount(),
])

dataset_report_snapshot = report.run(reference_data=reference, current_data=current)

```

[7] ✓ 0.0s

```

with open("dataset_report.html", "w", encoding="utf-8") as f:
    f.write(dataset_report_snapshot.get_html_str(as_iframe=False))

```

[10] ✓ 0.0s

```

report = Report(metrics=[
    DataDriftPreset(),
])

drift_report_snapshot = report.run(reference_data=reference, current_data=current)

with open("data_drift_report.html", "w", encoding="utf-8") as f:
    f.write(drift_report_snapshot.get_html_str(as_iframe=False))

```

[11] ✓ 0.5s

A streamlit appban a data set report.

Sajnos a reportokat nem sikerült jó meretben berakni.

Nem akartam iframe-mel szorakozni.

main page

Dataset Summary

Metric	Current	Reference
id column	None	None
target column	None	None
prediction column	None	None
date column	None	None
number of columns	14	
number of rows	6497	135
missing values	0	
categorical columns	3	
numeric columns	11	
text columns	0	
datetime columns	0	
empty columns	0	
constant columns	0	
almost constant features	0	
duplicated columns	0	
almost duplicated features	0	

A streamlit appban a data shift report.

main page

Dataset Drift

Dataset Drift is detected. Dataset drift detection

14

Columns

11

Drifted Columns

Data Drift Summary

Drift is detected for 78.571% of columns (11 out of 14).